EVM 스택 머신과 메모리 관리 분석

개념적 배경

Ethereum Virtual Machine(EVM)은 스택 기반 가상 머신으로, 모든 연산이 스택을 통해 이루어집니다. 또한 임시 데이터 저장을 위한 메모리와 영구 저장을 위한 스토리지, 그리고 트랜잭션 내에서만 유효한 임시 스토리지를 제공합니다.

설계 철학

1. **스택 기반 아키텍처**: 단순하고 검증 가능한 실행 모델
2. **메모리 계층 구조**: 속도와 비용의 균형
3. **오버플로우 방지**: 하드웨어 제약 내에서 안전한 실행
4. **가스 기반 제어**: 리소스 사용량에 따른 비용 부과

메모리 계층

1. **스택**: 연산 매개변수와 결과 저장 (최대 1024 항목)
2. **메모리**: 임시 데이터 저장 (바이트 배열, 동적 확장)
3. **스토리지**: 영구 데이터 저장 (키-값 매핑)
4. **임시 스토리지**: 트랜잭션 내 임시 데이터 (EIP-1153)

구현 분석

스택 구현

Stack 구조체

```go
type Stack struct {
data []uint256.Int // 256비트 정수 배열
}
```

스택 풀링 시스템

```go
var stackPool = sync.Pool{
New: func() interface{} {
return &Stack{data: make([]uint256.Int, 0, 16)} // 초기 용량 16
},
}

func newstack() *Stack {
return stackPool.Get().(*Stack)
}

func returnStack(s *Stack) {
s.data = s.data[:0] // 길이만 0으로 설정 (용량 유지)
stackPool.Put(s)
}
```

기본 스택 연산

```go
// 스택에 값 푸시
func (st *Stack) push(d *uint256.Int) {
// 1024 제한은 baseCheck에서 확인
st.data = append(st.data, *d)
}

// 스택에서 값 팝
func (st *Stack) pop() (ret uint256.Int) {
ret = st.data[len(st.data)-1]
st.data = st.data[:len(st.data)-1]
return
}

// 스택 최상단 값 참조 (팝하지 않음)
func (st *Stack) peek() *uint256.Int {
return &st.data[st.len()-1]
}

// n번째 값 참조 (0이 최상단)
func (st *Stack) Back(n  int) *uint256.Int {
return &st.data[st.len()-n-1]
}
```

스택 교환 연산 (SWAP1-SWAP16)

```
func (st *Stack) swap1() {
st.data[st.len()-2], st.data[st.len()-1] = st.data[st.len()-1], st.data[st.len()
}

func (st *Stack) swap2() {
st.data[st.len()-3], st.data[st.len()-1] = st.data[st.len()-1], st.data[st.len()
}

// ... swap3부터 swap16까지 동일한 패턴
```

스택 복제 연산 (DUP1-DUP16)

```
func (st *Stack) dup(n int) {
st.push(&st.data[st.len()-n])
}
```

스택 오버플로우/언더플로우 방지

스택 크기 검증

```
// 인터프리터에서 각 opcode 실행 전 검증
if sLen := stack.len(); sLen < operation.minStack {
return nil, &ErrStackUnderflow{stackLen: sLen, required: operation.minStack}
```

```
} else if sLen > operation.maxStack {
return nil, &ErrStackOverflow{stackLen: sLen, limit: operation.maxStack}
}
```

스택 제한 상수

```
const (
StackLimit uint64 = 1024 // 최대 스택 크기
)
```

에러 타입 정의

```
type ErrStackUnderflow struct {
stackLen int
required int
}

type ErrStackOverflow struct {
stackLen int
limit int
}
```

메모리 구현

Memory 구조체

```
type Memory struct {
store []byte  // 실제 메모리 데이터
lastGasCost  uint64  // 마지막 가스 비용 (증분 계산용)
}
```

메모리 풀링 시스템

```
var  memoryPool = sync.Pool{
New: func() any {
return &Memory{}
},
}

func  NewMemory() *Memory {
return  memoryPool.Get().(*Memory)
}

func (m *Memory) Free() {
// 16KB 이하의 메모리만 풀에 반환 (메모리 사용량 제한)
const  maxBufferSize = 16 << 10
if  cap(m.store) <= maxBufferSize {
m.store = m.store[:0]
```

```
m.lastGasCost = 0
memoryPool.Put(m)
}
}
```

메모리 확장

```
func (m *Memory) Resize(size uint64) {
if uint64(m.Len()) < size {
// 필요한 만큼 메모리 확장
m.store = append(m.store, make([]byte, size-uint64(m.Len()))...)
}
}
```

메모리 읽기/쓰기

```
// 메모리에서 데이터 복사본 반환
func (m *Memory) GetCopy(offset, size uint64) (cpy []byte) {
if size == 0 {
return nil
}
cpy = make([]byte, size)
copy(cpy, m.store[offset:offset+size])
return
```

```
}

// 메모리 포인터 직접 반환 (성능 최적화)
func (m *Memory) GetPtr(offset, size uint64) []byte {
if size == 0 {
return nil
}
return m.store[offset : offset+size]
}

// 메모리에 데이터 설정
func (m *Memory) Set(offset, size uint64, value []byte) {
if size > 0 {
if offset+size > uint64(len(m.store)) {
panic("invalid memory: store empty")
}
copy(m.store[offset:offset+size], value)
}
}

// 32바이트 값 설정 (최적화된 버전)
func (m *Memory) Set32(offset uint64, val *uint256.Int) {
if offset+32 > uint64(len(m.store)) {
```

```
panic("invalid memory: store empty")
}
val.PutUint256(m.store[offset:])
}
```

메모리 내부 복사 (MCOPY - EIP-5656)

```
func (m *Memory) Copy(dst, src, len uint64) {
if len == 0 {
return
}
copy(m.store[dst:], m.store[src:src+len])
}
```

메모리 가스 계산

메모리 확장은 이차 함수로 가스를 계산하여 대용량 메모리 사용을 억제합니다:

```
func memoryGasCost(mem *Memory, newMemSize uint64) (uint64, error) {
if newMemSize == 0 {
return 0, nil
}
// 최대 메모리 크기 제한 (오버플로우 방지)
if newMemSize > 0x1FFFFFFFE0 {
```

```
return 0, ErrGasUintOverflow
}
newMemSizeWords := toWordSize(newMemSize) // 32바이트 워드 단위
newMemSize = newMemSizeWords * 32
if newMemSize > uint64(mem.Len()) {
// 이차 함수: gas = 3 * words + words² / 512
square := newMemSizeWords * newMemSizeWords
linCoef := newMemSizeWords * params.MemoryGas // 3 * words
quadCoef := square / params.QuadCoeffDiv // words² / 512
newTotalFee := linCoef + quadCoef
// 증분만 계산 (이전 비용 제외)
fee := newTotalFee - mem.lastGasCost
mem.lastGasCost = newTotalFee
return fee, nil
}
return 0, nil
}
```

임시 스토리지 (EIP-1153)

EIP-1153에서 도입된 임시 스토리지는 트랜잭션 내에서만 유효한 키-값 저장소입니다.

TLOAD 연산

```
func opTload(pc *uint64, interpreter *EVMInterpreter, scope *ScopeContext) ([]b
```

```
loc := scope.Stack.peek()
hash := common.Hash(loc.Bytes32())
val := interpreter.evm.StateDB.GetTransientState(scope.Contract.Address(), hash)
loc.SetBytes(val.Bytes())
return nil, nil
}
```

TSTORE 연산

```
func opTstore(pc *uint64, interpreter *EVMInterpreter, scope *ScopeContext) ([]
if interpreter.readOnly {
return nil, ErrWriteProtection
}
loc := scope.Stack.pop()
val := scope.Stack.pop()
interpreter.evm.StateDB.SetTransientState(scope.Contract.Address(), loc.Bytes32(
return nil, nil
}
```

임시 스토리지 특징

- **트랜잭션 범위**: 트랜잭션 종료 시 자동 삭제
- **저렴한 비용**: SLOAD/SSTORE보다 훨씬 저렴 (100 가스)

- **재진입 안전**: 호출 간 데이터 공유 가능
- **가스 효율성**: 임시 데이터에 최적화

실행 흐름

스택 기반 연산 예시

산술 연산 (ADD)

```
초기 스택: [a, b]

ADD 실행:

1. x = stack.pop() // x = b, 스택: [a]

2. y = stack.peek() // y = a (참조), 스택: [a]

3. y.Add(&x, y) // y = a + b, 스택: [a+b]

최종 스택: [a+b]
```

메모리 저장 (MSTORE)

```
초기 스택: [offset, value]

MSTORE 실행:

1. mStart = stack.pop() // offset, 스택: [value]

2. val = stack.pop() // value, 스택: []

3. memory.Set32(mStart.Uint64(), &val) // 메모리[offset] = value

최종 스택: []
```

메모리 로드 (MLOAD)

```
초기 스택: [offset]

MLOAD 실행:

1. v = stack.peek() // offset (참조), 스택: [offset]

2. offset = v.Uint64() // offset 값 추출

3. v.SetBytes(memory.GetPtr(offset, 32)) // 메모리[offset] 읽기

최종 스택: [memory_value]
```

메모리 확장 과정

```
// 메모리 크기 계산이 필요한 연산 (MSTORE, MLOAD 등)

if  operation.memorySize != nil {

memSize, overflow := operation.memorySize(stack)

if  overflow {

return  nil, ErrGasUintOverflow

}

// 32바이트 워드 단위로 변환

if  memorySize, overflow = math.SafeMul(toWordSize(memSize), 32);  overflow {

return  nil, ErrGasUintOverflow

}

}


// 동적 가스 계산 (메모리 확장 비용 포함)

if  operation.dynamicGas != nil {
```

```
dynamicCost, err = operation.dynamicGas(evm, contract, stack, mem, memorySize)
if err != nil || contract.Gas < dynamicCost {
return nil, ErrOutOfGas
}
contract.Gas -= dynamicCost
}

// 메모리 확장 실행
if memorySize > 0 {
mem.Resize(memorySize)
}
```

성능 최적화

1. 메모리 풀링

스택과 메모리 객체를 재사용하여 GC 압박을 줄입니다:

```
// 스택 풀링
defer func() {
returnStack(stack) // 스택을 풀에 반환
mem.Free() // 메모리를 풀에 반환 (크기 제한 있음)
}()
```

2. 메모리 포인터 최적화

불필요한 복사를 피하기 위해 포인터를 직접 반환:

```
// 복사본 반환 (안전하지만 느림)
func (m *Memory) GetCopy(offset, size  uint64) []byte


// 포인터 반환 (빠르지만 주의 필요)
func (m *Memory) GetPtr(offset, size  uint64) []byte
```

3. 스택 연산 최적화

peek() 사용으로 불필요한 pop/push 방지:

```
// 비효율적
func  opAdd(pc *uint64, interpreter *EVMInterpreter, scope *ScopeContext) ([]byt
x := scope.Stack.pop()
y := scope.Stack.pop()
result := new(uint256.Int).Add(&x, &y)
scope.Stack.push(result)
return  nil, nil
}


// 효율적 (실제 구현)
func  opAdd(pc *uint64, interpreter *EVMInterpreter, scope *ScopeContext) ([]byt
```

```
x, y := scope.Stack.pop(), scope.Stack.peek() // y는 참조

y.Add(&x, y) // 결과를 y에 직접 저장

return nil, nil

}
```

4. 메모리 크기 제한

대용량 메모리 객체의 풀 반환을 제한하여 메모리 사용량 제어:

```
const maxBufferSize = 16 << 10 // 16KB

if cap(m.store) <= maxBufferSize {

memoryPool.Put(m) // 작은 메모리만 풀에 반환

}
```

에러 처리 메커니즘

스택 관련 에러

```
type ErrStackUnderflow struct {

stackLen int

required int

}


func (e *ErrStackUnderflow) Error() string {
```

```
return fmt.Sprintf("stack underflow (%d <=> %d)", e.stackLen, e.required)
}

type ErrStackOverflow struct {
stackLen int
limit int
}

func (e *ErrStackOverflow) Error() string {
return fmt.Sprintf("stack limit reached %d (%d)", e.stackLen, e.limit)
}
```

메모리 관련 에러

```
// 메모리 크기 오버플로우
if newMemSize > 0x1FFFFFFFE0 {
return 0, ErrGasUintOverflow
}

// 메모리 접근 범위 초과
if offset+size > uint64(len(m.store)) {
panic("invalid memory: store empty")
}
```

실제 사용 예시

간단한 계산 예시

```
바이트코드: PUSH1 0x10 PUSH1 0x20 ADD PUSH1 0x00 MSTORE
```

```
실행 과정:
1. PUSH1 0x10: 스택 [0x10]
2. PUSH1 0x20: 스택 [0x10, 0x20]
3. ADD: 스택 [0x30]
4. PUSH1 0x00: 스택 [0x30, 0x00]
5. MSTORE: 메모리[0x00] = 0x30, 스택 []
```

메모리 복사 예시

```
// Solidity 코드
bytes memory data = new bytes(64);
data[0] = 0x42;
```

```
// 생성되는 바이트코드 (단순화)
PUSH1 0x40  // 64 바이트
PUSH1 0x00  // offset 0
MSTORE // 메모리[0] = 64 (길이)
PUSH1 0x42  // 값 0x42
```

```
PUSH1 0x20  // offset 32 (데이터 시작)

MSTORE8 // 메모리[32] = 0x42
```

임시 스토리지 사용 예시

```
// Solidity에서 임시 스토리지 사용 (assembly)

assembly {

tstore(0x01, 0x42) // 임시 스토리지[1] = 0x42

let value := tload(0x01) // value = 0x42

}


// 생성되는 바이트코드

PUSH1 0x42

PUSH1 0x01

TSTORE // 임시 스토리지[1] = 0x42

PUSH1 0x01

TLOAD // 스택에 0x42 로드
```

관련 시스템

의존성 관계

- **Interpreter**: 스택과 메모리 생성/관리
- **StateDB**: 영구 스토리지와 임시 스토리지 인터페이스

- **Gas Calculator**: 메모리 확장 비용 계산
- **Contract**: 실행 컨텍스트 제공

상호작용 패턴

1. **Interpreter → Stack/Memory**: 연산 실행을 위한 데이터 조작
2. **Stack/Memory → Gas Calculator**: 리소스 사용량 계산
3. **Memory → StateDB**: 스토리지 연산 시 데이터 전달
4. **Contract → Stack/Memory**: 실행 컨텍스트 정보 제공

영향 범위

- **모든 EVM 연산**: 스택 기반 실행 모델
- **가스 소모**: 메모리 확장과 스택 연산 비용
- **성능**: 메모리 풀링과 최적화 기법
- **보안**: 오버플로우 방지와 리소스 제한

이 스택 머신과 메모리 관리 시스템은 EVM의 핵심 실행 환경을 제공하며, 안전성과 성능을 모두 고려한 정교한 설계를 보여줍니다.